

Course Service - Estructura MVC Completa

1. Estructura del Proyecto

```
course-service/
├── main.go           # Punto de entrada
├── config/
│   └── config.go     # Configuración
├── models/           # MODELS (MVC)
│   ├── course.go     # Modelo Course + Chapter
│   └── progress.go    # Modelos de progreso
├── controllers/      # CONTROLLERS (MVC)
│   ├── course_controller.go  # Endpoints principales de cursos
│   ├── chapter_controller.go # Endpoints de capítulos
│   ├── progress_controller.go # Endpoints de progreso
│   └── file_controller.go    # Endpoints de archivos/videos
├── services/         # SERVICES (Business Logic)
│   ├── course_service.go   # Lógica principal de cursos
│   ├── chapter_service.go  # Lógica de capítulos
│   ├── progress_service.go # Lógica de progreso
│   ├── file_service.go     # Manejo de archivos/videos
│   └── external_client.go  # Llamadas a otros servicios
├── routes/           # ROUTES/VIEWS (MVC)
│   └── routes.go        # Todas las rutas
├── utils/
│   ├── database.go       # Conexión BD + migraciones
│   ├── response.go       # Helpers respuestas
│   ├── errors.go         # Errores + middleware
│   └── file_utils.go     # Utilidades de archivos
├── static/           # Archivos estáticos
│   ├── videos/          # Videos por curso
│   ├── images/          # Imágenes de cursos
│   └── thumbnails/      # Miniaturas
├── Dockerfile
├── docker-compose.yml
├── .env.example
├── go.mod
└── README.md
```

2. Funcionalidades Migradas del Código Original

Del archivo `cursos.go` se migra:

-  Modelos `Curso` y `Capitulo` → `Course` y `Chapter`
-  Modelos `ProgresoUsuario` y `ProgresoCapitulo` → `UserProgress` y `ChapterProgress`
-  Función `getCursos()` → `CourseController.GetCourses()`
-  Función `getCursoById()` → `CourseController.GetCourseById()`
-  Función `createCurso()` → `CourseController.CreateCourse()`
-  Función `updateCurso()` → `CourseController.UpdateCourse()`
-  Función `deleteCurso()` → `CourseController.DeleteCourse()`
-  Función `getCapitulosByCurso()` → `ChapterController.GetChaptersByCourse()`
-  Función `createCapitulo()` → `ChapterController.CreateChapter()`
-  Función `updateCapitulo()` → `ChapterController.UpdateChapter()`
-  Función `deleteCapitulo()` → `ChapterController.DeleteChapter()`
-  Función `uploadVideo()` → `FileController.UploadVideo()`
-  Función `getVideo()` → `FileController.GetVideo()`
-  Función `deleteVideo()` → `FileController.DeleteVideo()`
-  Función `getProgresoUsuario()` → `ProgressController.GetUserProgress()`
-  Función `marcarCapituloCompletado()` → `ProgressController.MarkChapterCompleted()`
-  Función `guardarUltimoCapitulo()` → `ProgressController.SaveLastChapter()`

Nuevas funcionalidades agregadas:

-  Integración con Payment Service para verificar acceso
-  Endpoint para verificar acceso de usuario a curso
-  Eventos cuando se publica un curso
-  Validación de permisos por capítulo
-  Compresión automática de videos
-  Generación de thumbnails

3. Base de Datos Independiente

course_db (MySQL/MariaDB)

sql

-- *Tabla principal de cursos*

```
courses (  
  id, titulo, descripcion, contenido, precio, estado,  
  imagen_url, instructor_id, duracion_total, nivel,  
  created_at, updated_at, deleted_at  
)
```

-- *Tabla de capítulos*

```
chapters (  
  id, curso_id, titulo, descripcion, duracion, video_url,  
  video_nombre, orden, publicado, tipo_contenido,  
  created_at, updated_at, deleted_at  
)
```

-- *Tabla de progreso por usuario*

```
user_progress (  
  id, usuario_id, curso_id, porcentaje_total, ultimo_capitulo,  
  fecha_inicio, fecha_completado, tiempo_total,  
  created_at, updated_at  
)
```

-- *Tabla de progreso por capítulo*

```
chapter_progress (  
  id, usuario_id, curso_id, capitulo_id, completado,  
  progreso, tiempo_visto, fecha_completado,  
  created_at, updated_at  
)
```

-- *Tabla de categorías*

```
categories (  
  id, nombre, descripcion, slug, padre_id, activa,  
  created_at, updated_at  
)
```

4. APIs Expuestas

Cursos (Públicas y autenticadas)

GET	/api/courses	# Listar cursos públicos
GET	/api/courses/:id	# Obtener curso específico
POST	/api/courses	# Crear curso (auth)
PUT	/api/courses/:id	# Actualizar curso (auth)
DELETE	/api/courses/:id	# Eliminar curso (auth)
GET	/api/courses/:id/access	# Verificar acceso del usuario (auth)

Capítulos (Requieren autenticación)

GET /api/courses/:id/chapters # Capítulos de un curso
POST /api/chapters # Crear capítulo
PUT /api/chapters/:id # Actualizar capítulo
DELETE /api/chapters/:id # Eliminar capítulo
GET /api/chapters/:id/content # Contenido del capítulo (verifica acceso)

Progreso (Requieren autenticación)

GET /api/progress/course/:id # Progreso del usuario en curso
POST /api/progress/chapter/complete # Marcar capítulo completado
POST /api/progress/last-chapter # Guardar último capítulo visto
GET /api/progress/user/summary # Resumen de progreso del usuario

Archivos (Videos e imágenes)

POST /api/files/upload-video # Subir video (auth)
POST /api/files/upload-image # Subir imagen (auth)
GET /api/files/video/:courseId/:filename # Streaming de video (verifica acceso)
GET /api/files/image/:filename # Imágenes públicas
DELETE /api/files/:courseId/:filename # Eliminar archivo (auth)

Categorías

GET /api/categories # Listar categorías
POST /api/categories # Crear categoría (admin)
PUT /api/categories/:id # Actualizar categoría (admin)
DELETE /api/categories/:id # Eliminar categoría (admin)

Health & Info

GET /api/health # Health check
GET /api/stats # Estadísticas del servicio (admin)

5. Comunicación con Otros Servicios

Calls a otros microservicios:

Course Service → Payment Service:
GET /api/pagos/user/history (verificar pagos del usuario)
GET /api/pagos/:paymentId (verificar pago específico)

Course Service → User Service:
GET /api/users/:id (información del usuario)

Events emitidos:

- `course.created` (cuando se crea un curso)
- `course.published` (cuando se publica un curso)
- `course.updated` (cuando se actualiza un curso)
- `chapter.completed` (cuando un usuario completa un capítulo)
- `course.completed` (cuando un usuario completa todo el curso)

6. Características Especiales

Control de Acceso:

- Verificación automática con Payment Service
- Cursos gratuitos vs pagados
- Preview de capítulos sin pago
- Bloqueo de contenido premium

Manejo de Videos:

- Upload con validación de formato
- Streaming optimizado
- Compresión automática (opcional)
- Thumbnails generados automáticamente
- Soporte para múltiples resoluciones

Progreso Inteligente:

- Tracking automático de tiempo visto
- Sincronización en tiempo real
- Cálculo dinámico de porcentajes
- Recuperación de sesión

Performance:

- Cache de cursos populares
- Lazy loading de capítulos
- Compresión de respuestas
- CDN ready para archivos estáticos

7. Configuración por Ambiente

Variables de entorno:

```
bash
```

Database

```
DB_HOST=localhost  
DB_PORT=3306  
DB_USER=course_user  
DB_PASSWORD=course_password  
DB_NAME=course_db
```

Server

```
PORT=8003  
APP_ENV=development
```

External Services

```
PAYMENT_SERVICE_URL=http://localhost:8002  
USER_SERVICE_URL=http://localhost:8004
```

File Storage

```
UPLOAD_PATH=./static  
MAX_VIDEO_SIZE=100MB  
MAX_IMAGE_SIZE=5MB  
ALLOWED_VIDEO_FORMATS=mp4,webm,ogg  
ALLOWED_IMAGE_FORMATS=jpg,jpeg,png,webp
```

Cache

```
ENABLE_CACHE=true  
CACHE_TTL=300
```

CDN (opcional)

```
CDN_URL=https://cdn.example.com  
CDN_ENABLED=false
```

8. Compatibilidad 100% con Código Original

Mismas respuestas JSON:

- Estructura idéntica de cursos y capítulos
- Mismos campos y nomenclaturas
- Compatibilidad total con frontend existente

Mismas validaciones:

- Verificación de permisos
- Validación de archivos
- Control de estados de curso

Misma lógica de negocio:

- Cálculo de progreso
- Manejo de archivos
- Estados de publicación

¿Te gusta esta estructura? Mantiene toda la funcionalidad original pero bien organizada como microservicio independiente. Si apruebas, procedo a generar todo el código siguiendo la misma calidad del Payment Service.